From table 2.1 we can see that naïve choices would lead to a linear time requirement for at least one of the core operations, while a balanced tree would always guarantee a logarithmic worst-case. Although linear time is usually regarded as “feasible”, there is still a tremendous difference between logarithmic and linear: for a billion elements, the difference is between… 1 billion operations, and a few dozens. If each operation takes 1 millisecond, that means going from 11 days to less than a second.

Table 2.2 Performance for operations provided by PQs, broken down by underlaying data structure

Operation	Unsorted array	Sorted Array	Balanced Tree
Insert	O(1)	O(n)	O(log n)
Find-Minimum	O(1)*	O(1)	O(1)*
Delete-Minimum	O(n) $	O(1) §	O(log n)

(*) By saving an extra value with the minimum, and charging the cost of maintaining its value on insert and delete.

($) If we use a buffer to speed-up find-minimum, then we need to find the next minimum on delete (unfortunately, nothing comes for free: alternatively, we could have constant-time delete by giving up the buffer and swapping the deleted element with the last in the array, and linear time find-minimum).